

리눅스 네트워킹

LFCS 도메인 4 (배점 25%) — IP 설정, 방화벽, NAT, 리버스 프록시, 본딩, 시간 동기화, 진단

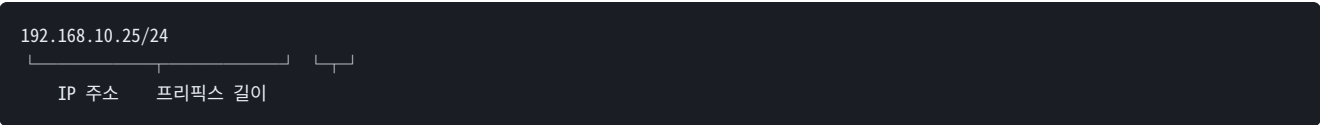
목차

- 1. 기초 개념 — IP, 서브넷, 포트
- 2. 인터페이스 확인과 임시 설정
- 3. 영구 설정 — Netplan / NetworkManager
- 4. 이름 해석 (DNS)
- 5. 라우팅
- 6. 네트워크 서비스 관리
- 7. 방화벽 — ufw / firewallld / nftables
- 8. 포트 리다이렉션과 NAT
- 9. 리버스 프록시와 로드 밸런서
- 10. 본딩과 팀ing
- 11. SSH 심화
- 12. 시간 동기화
- 13. 네트워크 진단 절차
- 14. 실습 체크리스트 / 자주 하는 실수

1. 기초 개념 — IP, 서브넷, 포트

1.1 IP 주소와 CIDR

IPv4 주소는 32비트를 8비트씩 끊어 표기합니다. 뒤에 붙는 /24 가 네트워크 부분의 비트 수입니다.



CIDR	서브넷 마스크	호스트 수	용도
/8	255.0.0.0	약 1,677만	대형 사설망
/16	255.255.0.0	약 65,534	중형 사설망, VPC
/24	255.255.255.0	254	가장 흔한 서브넷
/30	255.255.255.252	2	라우터 간 연결
/32	255.255.255.255	1	단일 호스트 지정

호스트 수가 $2^{(32-\text{프리픽스})} - 2$ 인 이유는 네트워크 주소(전부 0)와 브로드캐스트 주소(전부 1)를 쓸 수 없기 때문입니다.

사설 IP 대역

10.0.0.0/8	10.0.0.0 ~ 10.255.255.255
172.16.0.0/12	172.16.0.0 ~ 172.31.255.255
192.168.0.0/16	192.168.0.0 ~ 192.168.255.255
127.0.0.0/8	루프백 (자기 자신)
169.254.0.0/16	링크 로컬 (DHCP 실패 시 자동 할당)

서버에 169.254.x.x 주소가 붙어 있다면 **DHCP를 받지 못했다는 신호**입니다. 네트워크 케이블, VLAN 설정, DHCP 서버를 순서대로 의심하세요. 클라우드에서는 169.254.169.254 가 메타데이터 서비스 주소로 별도 사용됩니다.

1.2 포트

범위	구분	비고
0 ~ 1023	well-known	바인딩에 root 권한 필요
1024 ~ 49151	registered	일반 서비스
49152 ~ 65535	dynamic	클라이언트 임시 포트

```
22  SSH      80  HTTP      443 HTTPS
53   DNS     3306 MySQL    5432 PostgreSQL
6379 Redis   25  SMTP     123  NTP
9090 Prometheus 3000 Grafana 2049 NFS
```

1024 미만 포트를 열려면 root 권한이 필요합니다. 그래서 애플리케이션은 8080 같은 높은 포트에서 돌리고, 앞단의 리버스 프록시(80/443)가 넘겨주는 구조가 표준입니다. 또는 `setcap 'cap_net_bind_service=+ep'` 로 특정 바이너리에만 권한을 줍니다.

2. 인터페이스 확인과 임시 설정

2.1 ip 명령

`ifconfig`, `route`, `netstat` 은 폐기 예정(deprecated)입니다. 최신 배포판에는 아예 설치되어 있지 않은 경우가 많으니 `ip` 와 `ss` 를 쓰세요.

구형	현행
<code>ifconfig</code>	<code>ip addr</code>
<code>route -n</code>	<code>ip route</code>
<code>arp -a</code>	<code>ip neigh</code>
<code>netstat -tuln</code>	<code>ss -tuln</code>

```
ip addr          # 전체 인터페이스와 IP (축약: ip a)
ip addr show eth0 # 특정 인터페이스
ip -br addr      # 한 줄 요약 — 매우 유용
ip link          # 링크 계층 상태 (UP/DOWN, MAC)
ip route          # 라우팅 테이블
ip neigh         # ARP 캐시
ip -s link show eth0 # 송수신 통계, 에러 카운터
```

출력 읽기

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP
    link/ether 52:54:00:a1:b2:c3
    inet 192.168.10.25/24 brd 192.168.10.255 scope global eth0
```

UP 은 관리자가 활성화한 상태, LOWER_UP 은 **물리 링크가 실제로 연결됨**을 뜻합니다. UP 인데 LOWER_UP 이 없으면 케이블이나 스위치 포트 문제입니다.

2.2 임시 설정 (재부팅 시 사라짐)

```
sudo ip addr add 192.168.10.50/24 dev eth0
sudo ip addr del 192.168.10.50/24 dev eth0
sudo ip link set eth0 up
sudo ip link set eth0 down
sudo ip route add default via 192.168.10.1
sudo ip route add 10.0.0.0/8 via 192.168.10.254
```

원격 SSH로 접속한 상태에서 `ip link set eth0 down` 을 실행하면 **즉시 연결이 끊기고 되돌릴 방법이 없습니다**. 물리 콘솔이나 클라우드 시리얼 콘솔이 없다면 복구 불가능합니다.

안전장치: 위험한 네트워크 변경 전에 자동 복구를 예약해두세요.

```
sudo sh -c 'sleep 300 && reboot' & — 5분 안에 접속이 회복되지 않으면 재부팅되어 원래 설정으로 돌아갑니다. 성공하면 kill 로 취소합니다.
```

2.3 소켓 확인 — ss

```
ss -tuln          # TCP/UDP 리스닝 포트 (가장 자주 씬)
ss -tulnp         # 프로세스까지 (sudo 필요)
ss -t state established # 연결된 TCP 세션
ss -s            # 소켓 통계 요약
ss -tn dst 10.0.0.5 # 특정 대상과의 연결
```

옵션 의미: `-t` TCP, `-u` UDP, `-l` 리스닝, `-n` 숫자로 표시(이름 해석 생략), `-p` 프로세스.

"포트가 안 열린다"는 문의를 받으면 순서가 정해져 있습니다. ① `ss -tulnp` 로 서비스가 실제로 리스닝 중인지 ② 바인딩 주소가 `127.0.0.1` 인지 `0.0.0.0` 인지 ③ 방화벽 ④ 보안그룹/네트워크 ACL.

②번이 특히 흔합니다. `127.0.0.1:3306` 으로 리스닝 중이면 외부에서는 절대 접속할 수 없습니다.

3. 영구 설정 — Netplan / NetworkManager

배포판마다 방식이 다릅니다. 어느 것이 쓰이는지 먼저 확인해야 합니다.

배포판	설정 방식	경로
Ubuntu 18.04+	Netplan (YAML)	<code>/etc/netplan/*.yaml</code>
RHEL 8/9, Rocky	NetworkManager	<code>nmcli</code> , <code>/etc/NetworkManager/</code>
Debian	ifupdown 또는 Netplan	<code>/etc/network/interfaces</code>

3.1 Netplan (Ubuntu)

```
sudo vim /etc/netplan/01-netcfg.yaml
```

고정 IP

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: false
      addresses:
        - 192.168.10.25/24
      routes:
        - to: default
          via: 192.168.10.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

DHCP

```
network:
  version: 2
  ethernets:
    eth0:
      dhcp4: true
```

```
sudo netplan try    # 적용 후 120초 내 확인 없으면 자동 롤백 — 반드시 이걸 먼저
sudo netplan apply  # 즉시 적용
sudo netplan get    # 현재 설정 확인
```

YAML은 들여쓰기와 탭에 매우 민감합니다. 탭 문자를 쓰면 파싱에 실패하고, 실패하면 네트워크가 올라오지 않습니다. 반드시 스페이스만 쓰고, 원격에서는 `netplan apply` 대신 **`netplan try`** 를 쓰세요. 문제가 있으면 자동으로 되돌립니다.

파일 퍼미션도 확인하세요. Netplan 설정에 자격증명이 들어갈 수 있어 `chmod 600` 이 권장됩니다.

3.2 nmcli (RHEL 계열)

```
nmcli device status      # 장치 상태
nmcli connection show    # 연결 프로필 목록
nmcli connection show "System eth0" # 상세

# 고정 IP 프로필 생성
sudo nmcli connection add type ethernet con-name static-eth0 ifname eth0 \
  ipv4.method manual \
  ipv4.addresses 192.168.10.25/24 \
```

```

ip4.gateway 192.168.10.1 \
ip4.dns "8.8.8.8 1.1.1.1"

# 기존 프로필 수정
sudo nmcli connection modify static-eth0 ip4.addresses 192.168.10.30/24

# 적용
sudo nmcli connection up static-eth0
sudo nmcli device reapply eth0

```

nmcli 로 만든 설정은 즉시 파일로 저장되므로 재부팅 후에도 유지됩니다. ip addr add 는 메모리에만 적용되어 재부팅 시 사라진다는 점이 결정적 차이입니다. 시험이나 실무에서 "영구 설정"을 요구하면 반드시 nmcli 또는 netplan을 써야 합니다.

3.3 호스트명

```

hostnamectl                # 현재 정보
sudo hostnamectl set-hostname web01  # 영구 변경

```

/etc/hosts 에도 새 호스트명을 등록해두어야 이름 해석 지연이 생기지 않습니다.

4. 이름 해석 (DNS)

4.1 해석 순서

/etc/nsswitch.conf 의 hosts: 줄이 순서를 결정합니다. 보통 /etc/hosts → DNS 순입니다.

```

cat /etc/nsswitch.conf | grep hosts
# hosts: files dns

```

4.2 /etc/hosts

```

127.0.0.1    localhost
192.168.10.25 web01 web01.internal
192.168.10.30 db01

```

DNS보다 먼저 조회되므로 테스트나 임시 우회에 유용합니다. 다만 서버가 많아지면 관리가 불가능하니 임시 수단으로만 쓰세요.

4.3 DNS 서버 설정

```
cat /etc/resolv.conf
```

최신 Ubuntu에서 /etc/resolv.conf 는 systemd-resolved 가 관리하는 [심볼릭 링크](#)입니다. 직접 편집해도 재부팅하면 덮어써집니다. Netplan의 nameservers 항목이나 nmcli 로 설정하는 것이 올바른 방법입니다.

```

resolvectl status      # systemd-resolved 상태와 실제 사용 중인 DNS
resolvectl flush-caches # DNS 캐시 비우기

```

4.4 조회 도구

```

dig example.com          # 상세 조회 (권장)
dig +short example.com   # 결과만
dig @8.8.8.8 example.com # 특정 DNS 서버로 조회
dig example.com MX       # 메일 레코드
dig -x 8.8.8.8           # 역방향 조회

```

```
host example.com          # 간단 조회
getent hosts example.com  # nsswitch 순서를 그대로 따름
```

dig 와 getent hosts 의 차이가 중요합니다. dig 는 **DNS만** 조회하고 /etc/hosts 를 무시합니다. getent hosts 는 시스템이 실제로 쓰는 경로를 그대로 따릅니다. "dig로는 되는데 애플리케이션은 못 찾는다"면 getent 로 확인하세요.

5. 라우팅

```
ip route
# default via 192.168.10.1 dev eth0 proto static
# 192.168.10.0/24 dev eth0 proto kernel scope link src 192.168.10.25
```

패킷은 가장 구체적인 경로(프리픽스가 긴 것)부터 매칭됩니다. default (= 0.0.0.0/0)는 어디에도 해당하지 않을 때의 최후 경로입니다.

```
sudo ip route add 10.20.0.0/16 via 192.168.10.254 dev eth0 # 정적 경로 추가
sudo ip route del 10.20.0.0/16
ip route get 8.8.8.8 # 이 목적지로 어떤 경로가 선택되는지 확인
```

ip route get 은 진단에서 매우 유용합니다. 라우팅 테이블을 눈으로 해석할 필요 없이 커널이 실제로 어떤 경로와 출발 IP를 고르는지 알려줍니다. 인터페이스가 여러 개일 때 필수입니다.

IP 포워딩

리눅스를 라우터/NAT 게이트웨이로 쓰려면 활성화해야 합니다.

```
sysctl net.ipv4.ip_forward # 현재 값 확인
sudo sysctl -w net.ipv4.ip_forward=1 # 임시 적용

# 영구 설정
echo "net.ipv4.ip_forward=1" | sudo tee /etc/sysctl.d/99-forward.conf
sudo sysctl --system
```

6. 네트워크 서비스 관리

```
systemctl status nginx # 상태 확인
sudo systemctl start nginx # 시작
sudo systemctl stop nginx # 중지
sudo systemctl restart nginx # 재시작 (연결 끊김)
sudo systemctl reload nginx # 설정만 재적용 (무중단)
sudo systemctl enable nginx # 부팅 시 자동 시작
sudo systemctl disable nginx # 자동 시작 해제
sudo systemctl enable --now nginx # 활성화 + 즉시 시작
```

restart 와 reload 의 차이가 실무에서 중요합니다. restart 는 프로세스를 죽였다 살리므로 진행 중인 연결이 끊깁니다. reload 는 설정 파일만 다시 읽어 무중단입니다. 운영 중인 웹 서버 설정을 바꿨다면 reload 를 쓰세요.

다만 reload 로 반영되지 않는 항목(리스닝 포트 변경 등)이 있으니, 반영이 안 되면 restart 가 필요합니다.

설정 검증 습관

```
sudo nginx -t # nginx 설정 문법 검사
sudo sshd -t # sshd 설정 검사
sudo systemctl reload nginx # 검사 통과 후 적용
```

검증 없이 재시작했다가 문법 오류로 서비스가 안 올라오는 사고를 막는 기본 절차입니다.

7. 방화벽 — ufw / firewalld / nftables

세 계층이 있습니다. 실제 필터링은 커널의 netfilter가 하고, iptables / nftables 가 그 규칙을 다루며, ufw / firewalld 는 그 위의 사용자 친화적 관리 도구입니다.

7.1 ufw (Ubuntu)

```
sudo ufw status verbose
sudo ufw default deny incoming    # 기본 정책: 인바운드 차단
sudo ufw default allow outgoing   # 아웃바운드 허용

sudo ufw allow 22/tcp              # SSH 허용
sudo ufw allow ssh                 # 서비스명으로도 가능
sudo ufw allow 80,443/tcp          # 여러 포트
sudo ufw allow from 192.168.10.0/24 to any port 3306 # 출발지 제한
sudo ufw deny 23                  # 차단

sudo ufw enable                   # 활성화
sudo ufw status numbered          # 번호와 함께 확인
sudo ufw delete 3                 # 번호로 삭제
```

SSH 규칙을 먼저 추가하고 방화벽을 켜세요. ufw enable 을 SSH 허용 없이 실행하면 그 즉시 접속이 끊기고 다시 들어올 수 없습니다. 순서:

① sudo ufw allow 22/tcp → ② sudo ufw enable → ③ 기존 세션을 유지한 채 새 터미널로 접속 확인.

7.2 firewalld (RHEL 계열)

firewalld는 존(zone) 개념으로 동작합니다. 인터페이스마다 다른 신뢰 수준을 부여할 수 있습니다.

```
sudo firewall-cmd --state
sudo firewall-cmd --get-active-zones
sudo firewall-cmd --list-all      # 기본 존의 전체 규칙

sudo firewall-cmd --add-service=http --permanent # 서비스 허용
sudo firewall-cmd --add-port=8080/tcp --permanent # 포트 허용
sudo firewall-cmd --remove-port=8080/tcp --permanent
sudo firewall-cmd --reload         # 영구 규칙 적용

sudo firewall-cmd --zone=internal --add-source=192.168.10.0/24 --permanent
```

--permanent 없이 추가한 규칙은 즉시 적용되지만 재부팅 시 사라집니다. 반대로 --permanent 만 쓰면 파일에는 저장되지만 --reload 전까지 적용되지 않습니다.

안전한 패턴: 먼저 --permanent 없이 추가해 동작을 확인한 뒤, --permanent 로 다시 추가하고 --reload . 잘못된 규칙이라도 재부팅하면 사라지므로 잠길 위험이 줄어듭니다.

7.3 nftables / iptables

```
sudo nft list ruleset             # 현재 전체 규칙
sudo iptables -L -n -v            # 레거시 문법 조회
sudo iptables -t nat -L -n        # NAT 테이블
```

최신 배포판은 내부적으로 nftables를 쓰고 iptables 명령은 호환 래퍼인 경우가 많습니다. 신규 구성이라면 nftables를 배우는 편이 낫습니다.

8. 포트 리다이렉션과 NAT

8.1 NAT의 두 방향

종류	방향	용도
SNAT / MASQUERADE	출발지 주소 변경	사설망 → 인터넷 (게이트웨이)
DNAT / 포트 포워딩	목적지 주소 변경	인터넷 → 내부 서버

8.2 firewallld로 포트 리다이렉션

```
# 외부 8080 → 내부 80
sudo firewall-cmd --add-forward-port=port=8080:proto=tcp:toport=80 --permanent

# 외부 8080 → 다른 서버의 80
sudo firewall-cmd --add-forward-port=port=8080:proto=tcp:toport=80:toaddr=192.168.10.30 --permanent

sudo firewall-cmd --add-masquerade --permanent # SNAT 활성화
sudo firewall-cmd --reload
```

8.3 iptables로 직접

```
# DNAT — 들어오는 8080을 내부 서버 80으로
sudo iptables -t nat -A PREROUTING -p tcp --dport 8080 \
-j DNAT --to-destination 192.168.10.30:80

# MASQUERADE — 내부망이 eth0으로 나갈 때 출발지 변환
sudo iptables -t nat -A POSTROUTING -s 192.168.10.0/24 -o eth0 -j MASQUERADE
```

NAT/포워딩은 **net.ipv4.ip_forward=1** 이 없으면 동작하지 않습니다. 5장의 sysctl 설정을 먼저 확인하세요. 규칙은 맞는데 패킷이 안 넘어간다면 십중팔구 이것입니다.

그리고 iptables 규칙은 **재부팅 시 사라집니다**. iptables-persistent 패키지를 설치하거나 netfilter-persistent save 로 저장해야 합니다.

8.4 SSH 터널링

방화벽을 건드리지 않고 임시로 포트를 노출할 때 유용합니다.

```
# 로컬 포워딩: 내 8080 → 원격 서버의 3306
ssh -L 8080:localhost:3306 user@server

# 원격 포워딩: 원격의 9000 → 내 로컬 3000
ssh -R 9000:localhost:3000 user@server

# 동적 (SOCKS 프록시)
ssh -D 1080 user@server
```

운영 DB에 직접 접속이 막혀 있을 때, 점프 서버를 경유하는 -L 터널이 표준 방식입니다. DB 포트를 외부에 노출하지 않으면서 로컬 톨로 접속할 수 있습니다.

9. 리버스 프록시와 로드 밸런서

9.1 개념

포워드 프록시는 클라이언트를 대신해 나가고, 리버스 프록시는 서버를 대신해 요청을 받습니다.

```
[클라이언트] → [리버스 프록시] → [백엔드 서버들]
                ↑
            80/443만 노출
            백엔드는 내부망에 숨김
```

리버스 프록시를 두는 이유

- TLS 종료 — 인증서를 한 곳에서만 관리
- 백엔드 은닉 — 애플리케이션 서버를 외부에 노출하지 않음
- 로드 밸런싱 — 여러 백엔드로 분산
- 캐싱·압축 — 정적 자원 처리
- 단일 진입점 — 로깅, 레이트 리밋, WAF 적용 지점

9.2 Nginx 리버스 프록시

```
sudo vim /etc/nginx/sites-available/app
```

```
server {
    listen 80;
    server_name app.example.com;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP      $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

X-Forwarded-For 헤더를 반드시 넣으세요. 이게 없으면 백엔드 애플리케이션의 로그에 모든 접속자가 프록시 IP로 기록됩니다. 실제 클라이언트 IP 기반의 레이트 리밋이나 접근 제어가 전부 무력화됩니다. 장애 분석에서도 치명적입니다.

9.3 로드 밸런서로 확장

```
upstream backend {
    server 192.168.10.31:8080 weight=3;
    server 192.168.10.32:8080;
    server 192.168.10.33:8080 backup;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}
```

방식	설정	특징
라운드로빈	기본값	순서대로 분배

가중치	weight=3	서버 성능이 다를 때
최소 연결	least_conn;	처리 시간이 들쭉날쭉할 때
IP 해시	ip_hash;	같은 클라이언트를 같은 서버로 (세션 유지)

backup 은 다른 서버가 모두 실패했을 때만 사용됩니다.

```
sudo ln -s /etc/nginx/sites-available/app /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

10. 본딩과 팀링

10.1 목적

여러 물리 인터페이스를 하나의 논리 인터페이스로 묶습니다. 목적은 두 가지입니다.

- **가용성** — NIC나 스위치 포트 하나가 죽어도 통신 유지
- **대역폭** — 여러 링크를 합쳐 처리량 증가

10.2 모드

모드	이름	특징	스위치 설정
0	balance-rr	순서대로 분배. 패킷 순서 뒤바뀔 수 있음	필요
1	active-backup	하나만 활성, 나머지는 대기. 가장 안전	불필요
4	802.3ad (LACP)	표준 링크 집성. 대역폭 + 이중화	필요
6	balance-alb	송수신 부하 분산	불필요

스위치를 건드릴 수 없는 환경이면 **mode 1 (active-backup)**을, 스위치에서 LACP를 설정할 수 있으면 **mode 4**를 선택하는 것이 실무 기준입니다. mode 4를 스위치 설정 없이 쓰면 통신이 아예 안 됩니다.

10.3 Netplan 구성 예

```
network:
  version: 2
  ethernets:
    eth0: {dhcp4: no}
    eth1: {dhcp4: no}
  bonds:
    bond0:
      interfaces: [eth0, eth1]
      addresses: [192.168.10.25/24]
      routes:
        - to: default
          via: 192.168.10.1
      parameters:
        mode: active-backup
        primary: eth0
        mii-monitor-interval: 100
```

```
cat /proc/net/bonding/bond0 # 본딩 상태와 활성 슬레이브 확인
```


11. SSH 심화

11.1 키 인증 설정

```
ssh-keygen -t ed25519 -C "jihoon@laptop"      # 키 생성
ssh-copy-id jihoon@192.168.10.25             # 공개키 전송

# 수동으로 할 경우
cat ~/.ssh/id_ed25519.pub | ssh user@host 'mkdir -p ~/.ssh && cat >> ~/.ssh/authorized_keys'
```

필수 퍼미션

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
chmod 600 ~/.ssh/id_ed25519
```

퍼미션이 느슨하면 SSH가 **조용히 키 인증을 거부**하고 비밀번호를 묻습니다. "키를 등록했는데 비밀번호를 물어본다"면 가장 먼저 이걸 확인하세요. 홈 디렉터리 자체가 그룹 쓰기 가능(775)이어도 거부됩니다.

원인 파악은 `ssh -vvv user@host` 로 클라이언트 로그를, 서버에서는 `journalctl -u ssh -f` 로 확인합니다.

11.2 서버 설정 강화

```
sudo vim /etc/ssh/sshd_config
```

```
Port 22
PermitRootLogin no          # root 직접 로그인 차단
PasswordAuthentication no   # 키 인증만 허용
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers jihoon minseo    # 접속 가능 사용자 화이트리스트
ClientAliveInterval 300
```

```
sudo sshd -t                # 문법 검사 — 반드시 먼저
sudo systemctl reload sshd
```

SSH 설정을 바꾼 뒤 **현재 세션을 절대 닫지 마세요**. 새 터미널로 접속이 되는지 확인한 다음에 기존 세션을 정리합니다. 설정 오류로 SSH가 죽으면 원격 복구가 불가능합니다.

11.3 클라이언트 설정

```
vim ~/.ssh/config
```

```
Host web01
  HostName 192.168.10.25
  User jihoon
  Port 22
  IdentityFile ~/.ssh/id_ed25519

Host db01
  HostName 10.0.5.30
  User jihoon
  ProxyJump web01          # 점프 서버 경유
```

이제 `ssh web01` 만으로 접속됩니다. `ProxyJump` 는 배스천 호스트를 거쳐야 하는 환경에서 필수입니다.

12. 시간 동기화

12.1 왜 중요한가

서버 시계가 어긋나면 다음이 전부 깨집니다.

- 여러 서버의 로그를 시간순으로 대조할 수 없음 — 장애 분석 불가
- TLS 인증서 유효기간 검증 실패
- Kerberos, JWT, TOTP 등 시간 기반 인증 실패
- 분산 시스템의 순서 판단 오류

12.2 timedatectl

```
timedatectl                                # 현재 상태
timedatectl list-timezones | grep Seoul
sudo timedatectl set-timezone Asia/Seoul   # 타임존 변경
sudo timedatectl set-ntp true               # NTP 동기화 활성화
```

출력에서 확인할 항목: `System clock synchronized: yes` 와 `NTP service: active` . 둘 다 yes/active여야 정상입니다.

12.3 chrony

```
sudo apt install chrony                    # 또는 dnf install chrony
sudo vim /etc/chrony/chrony.conf
```

```
pool kr.pool.ntp.org iburst
server time.bora.net iburst
```

```
sudo systemctl enable --now chronyd
chronyc sources -v          # 동기화 소스 상태
chronyc tracking             # 현재 오차와 보정 상태
```

`chronyc sources` 출력에서 소스 앞의 `^*` 가 **현재 동기화 중인 서버**입니다. `^-` 는 후보, `^?` 는 도달 불가입니다. 전부 `^?` 라면 방화벽에서 UDP 123이 막혀 있는지 확인하세요.

타임존은 Asia/Seoul 로 두되, **로그는 UTC로 남기는** 구성이 다국가 서비스에서 일반적입니다. 서버 시계 자체는 항상 UTC로 두고 표시만 로컬로 하는 방식도 많이 씁니다.

13. 네트워크 진단 절차

13.1 계층별로 좁혀 들어가기

무작정 명령을 던지지 말고 아래에서 위로 올라가며 확인합니다.

단계	확인 사항	명령
1. 물리/링크	인터페이스가 UP인가	<code>ip -br link</code>
2. IP	주소가 붙었는가	<code>ip -br addr</code>
3. 게이트웨이	기본 경로가 있는가	<code>ip route</code>
4. 로컬 도달	게이트웨이에 닿는가	<code>ping 게이트웨이</code>
5. 외부 도달	인터넷에 닿는가	<code>ping 8.8.8.8</code>
6. DNS	이름 해석이 되는가	<code>dig example.com</code>
7. 포트	대상 포트가 열렸는가	<code>nc -zv host 443</code>
8. 애플리케이션	응답이 오는가	<code>curl -v</code>

핵심 분기점은 5번과 6번 사이입니다. ping 8.8.8.8 은 되는데 ping google.com 이 안 되면 네트워크는 정상이고 **DNS만 문제**입니다. 이 한 가지 구분만 으로 원인 범위가 절반으로 줄어듭니다.

13.2 도구별 용도

```
ping -c 4 8.8.8.8          # 도달성 (ICMP 차단 환경에선 무의미할 수 있음)
traceroute example.com      # 경로 추적
mtr example.com             # ping + traceroute 실시간 결합 (강력)

nc -zv 192.168.10.30 3306    # 포트 열림 확인
curl -v https://example.com  # HTTP 상세
curl -I https://example.com  # 헤더만
curl -o /dev/null -s -w "%{time_total}\n" https://example.com # 응답 시간

sudo tcpdump -i eth0 port 443 -nn      # 패킷 캡처
sudo tcpdump -i eth0 host 10.0.0.5 -w cap.pcap # 파일로 저장
```

많은 클라우드와 방화벽이 ICMP를 차단합니다. ping 이 안 된다고 해서 서버가 죽은 게 아닙니다. 서비스 도달성은 `nc -zv` 나 `curl` 로 판단하세요.

tcpdump 읽는 요령

TCP 연결이 안 될 때 캡처를 보면 원인이 드러납니다.

- **SYN만 반복** → 응답 없음. 방화벽에서 DROP 중이거나 대상이 죽음
- **SYN → RST** → 대상이 명시적으로 거부. 포트가 안 열림 또는 방화벽 REJECT
- **SYN → SYN-ACK → ACK** → 연결 자체는 성공. 문제는 애플리케이션 계층

14. 실습 체크리스트 / 자주 하는 실수

14.1 실습 체크리스트

1. `ip -br addr` 과 `ip -br link` 로 인터페이스 상태 확인
2. `ip addr add` 로 임시 IP를 추가하고 재부팅 후 사라지는 것 확인

3. Netplan(또는 nmcli)으로 고정 IP를 설정하고 `netplan try` 로 안전하게 적용
4. `ss -tulnp` 로 리스닝 포트와 프로세스 매핑 확인
5. 서비스를 127.0.0.1 에만 바인딩한 뒤 외부 접속이 실패하는 것 재현
6. `ip route get 8.8.8.8` 로 선택되는 경로와 출발 IP 확인
7. ufw(또는 firewallld)로 SSH를 먼저 허용한 뒤 방화벽 활성화
8. 특정 출발지 대역에서만 DB 포트를 허용하는 규칙 작성
9. `net.ipv4.ip_forward` 를 끈 상태와 켜 상태에서 포워딩 동작 비교
10. Nginx 리버스 프록시를 구성하고 X-Forwarded-For 유무에 따른 백엔드 로그 차이 확인
11. upstream에 서버 두 대를 넣고 한 대를 내렸을 때 동작 확인
12. `ssh -L` 터널로 원격 DB 포트에 로컬 접속
13. SSH 키 퍼미션을 644 로 바꿔 인증이 거부되는 것 확인 후 복구
14. `timedatectl` 로 타임존 변경, `chronyc sources` 로 동기화 확인
15. `tcpdump` 로 SYN/RST 패턴을 직접 관찰 (닫힌 포트에 접속 시도)

14.2 자주 하는 실수

실수	결과	올바른 방법
SSH 허용 전 방화벽 활성화	즉시 접속 차단	규칙 먼저 → 활성화 → 새 세션 확인
원격에서 <code>netplan apply</code>	오류 시 복구 불가	<code>netplan try</code> (자동 롤백)
Netplan YAML에 탭 사용	파싱 실패, 네트워크 미기동	스페이스만 사용
<code>/etc/resolv.conf</code> 직접 편집	재부팅 시 초기화	Netplan/nmcli로 설정
firewalld <code>--permanent</code> 후 reload 누락	규칙 미적용	<code>--reload</code> 실행
NAT 규칙만 넣고 <code>ip_forward</code> 누락	패킷 전달 안 됨	<code>sysctl net.ipv4.ip_forward=1</code>
iptables 규칙 저장 안 함	재부팅 시 소실	<code>netfilter-persistent save</code>
프록시에 X-Forwarded-For 누락	실제 클라이언트 IP 유실	<code>proxy_set_header</code> 추가
본딩 mode 4를 스위치 설정 없이	통신 불가	mode 1 또는 스위치 LACP 설정
SSH 설정 후 세션 종료	재접속 불가	<code>sshd -t</code> + 기존 세션 유지
ICMP 차단 환경에서 ping으로 판단	오진	<code>nc -zv</code> / <code>curl</code>
운영 서버에 <code>restart</code> 남용	연결 끊김	가능하면 <code>reload</code>

학습용으로 새로 정리한 자료입니다. 명령과 설정 문법은 배포판·버전에 따라 다를 수 있으니 실제 적용 전 `man` 페이지와 공식 문서로 확인하시기 바랍니다.